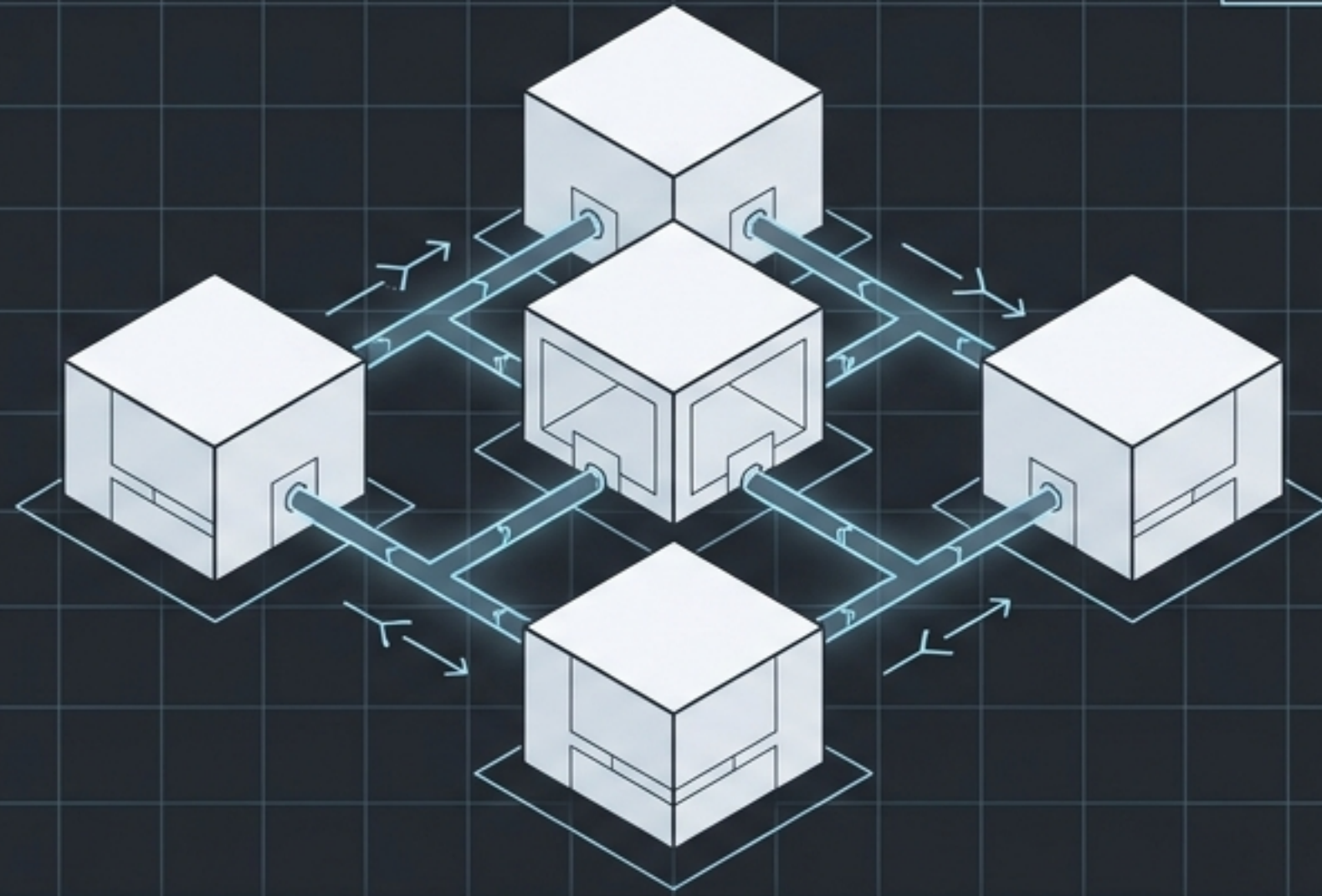




Cetak Biru UTS: Sistem Pengelolaan Gaji Karyawan

Panduan Arsitektur, Logika Modular, dan Analisis Kompleksitas

5 Pilar Spesifikasi Sistem

Input

Menerima jumlah dan nominal gaji karyawan (bilangan bulat, jutaan rupiah).

Searching

Mencari nominal gaji spesifik yang diinputkan pengguna.

Analisis

Mencetak audit kompleksitas sistem di layar terminal.

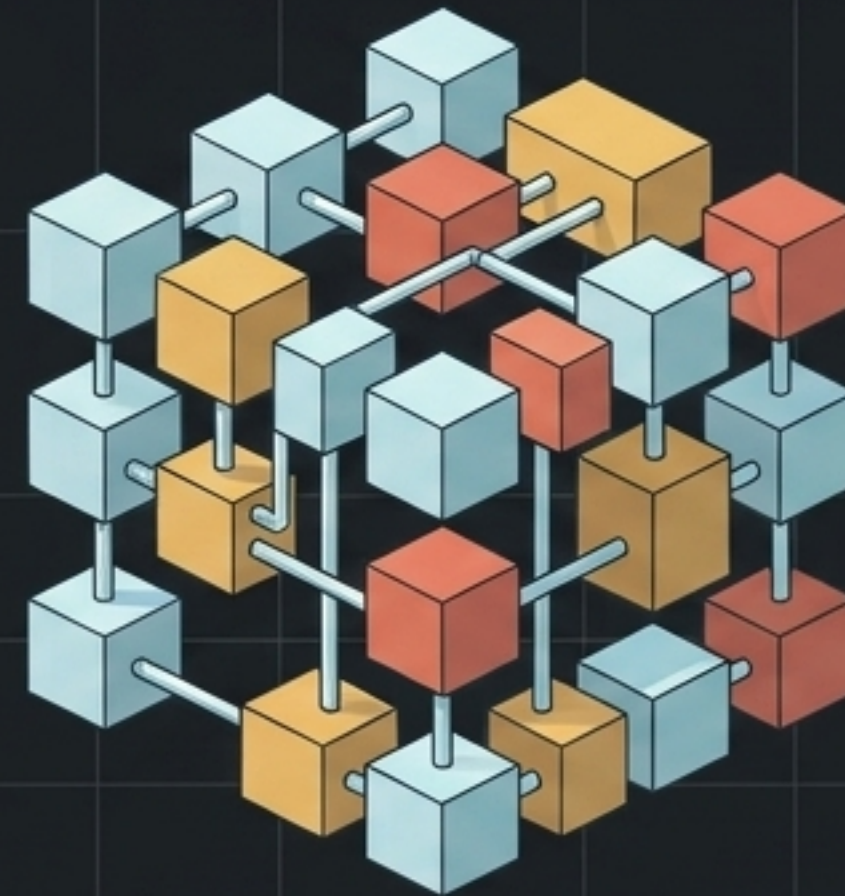
Sorting

Mengurutkan data dari kecil ke besar secara sekuensial.

Output

Menampilkan hasil urutan, posisi pencarian, dan metrik kinerja (jumlah langkah).

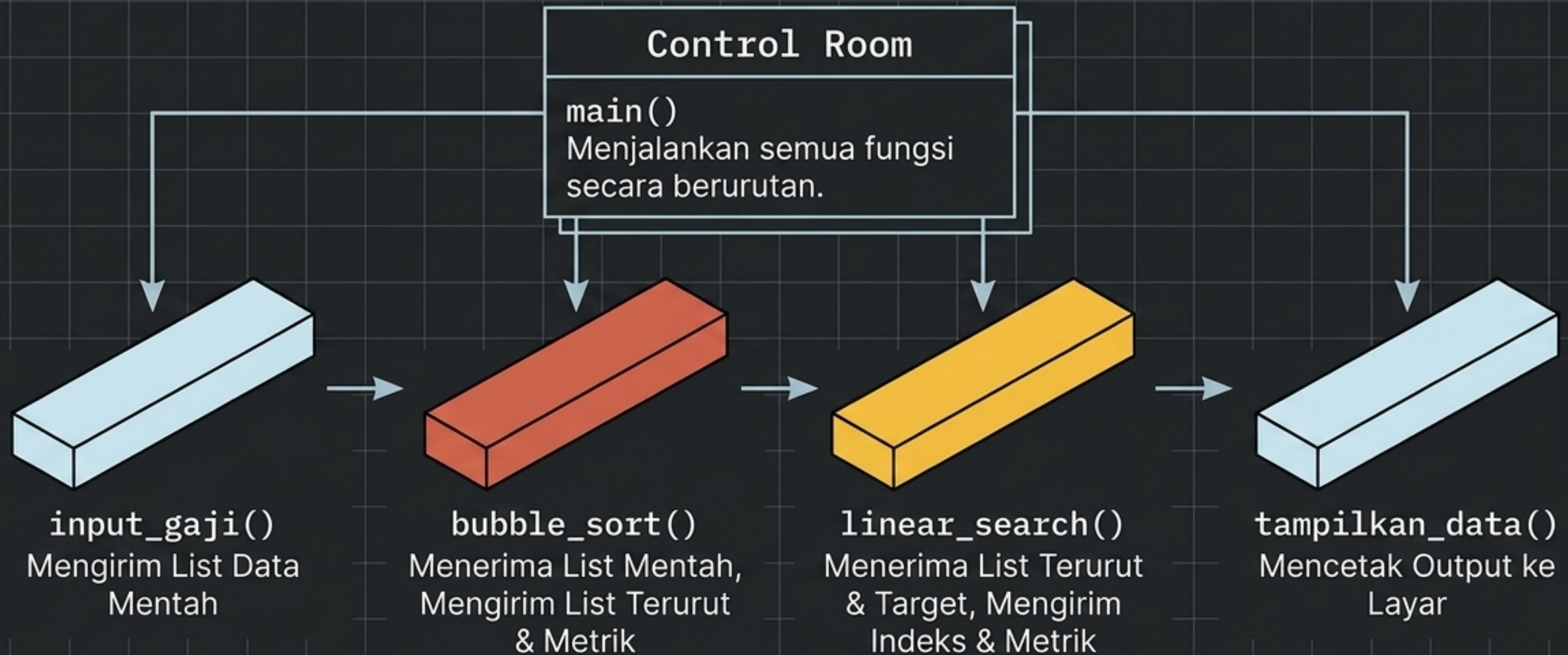
Dekomposisi Sistem: Dari Monolitik ke Modular



Aturan mutlak ujian: Jangan gabungkan logika pengurutan, pencarian, dan input dalam satu blok kode raksasa.

Single Responsibility: Satu fungsi, tepat satu tugas logis. Setiap sub-masalah sistem gaji harus menjadi entitas terpisah.

Arsitektur Pipeline Data Gaji



Anatomi I/O: Batasan dan Tanggung Jawab

```
input_gaji()
```

Parameter: Tidak ada (Kosong)

Proses: Meminta input jumlah karyawan (n) lalu melakukan looping untuk meminta nominal gaji per karyawan.

Return: Menghasilkan list berisi data gaji murni.

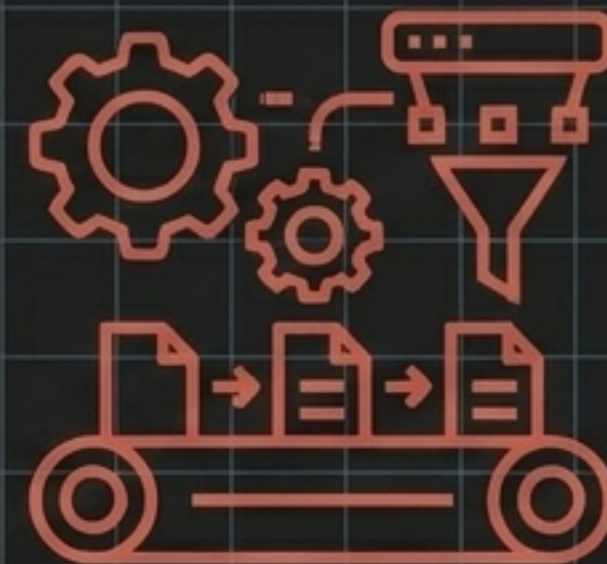
```
tampilkan_data(arr, urutan_asal=None)
```

Parameter: List gaji (arr).

Proses: Melakukan iterasi untuk mencetak isi list dengan rapi ke layar. Tidak boleh ada logika manipulasi data di sini.

Return: Tidak mengembalikan nilai struktural (Void).

Mesin Pengurut: Spesifikasi Bubble Sort



```
def bubble_sort(arr):
```

Tujuan: Mengurutkan gaji dari nilai terkecil ke terbesar (Ascending).

Output Ganda: Wajib mengembalikan dua nilai: list yang sudah diurutkan, dan total jumlah_perbandingan.

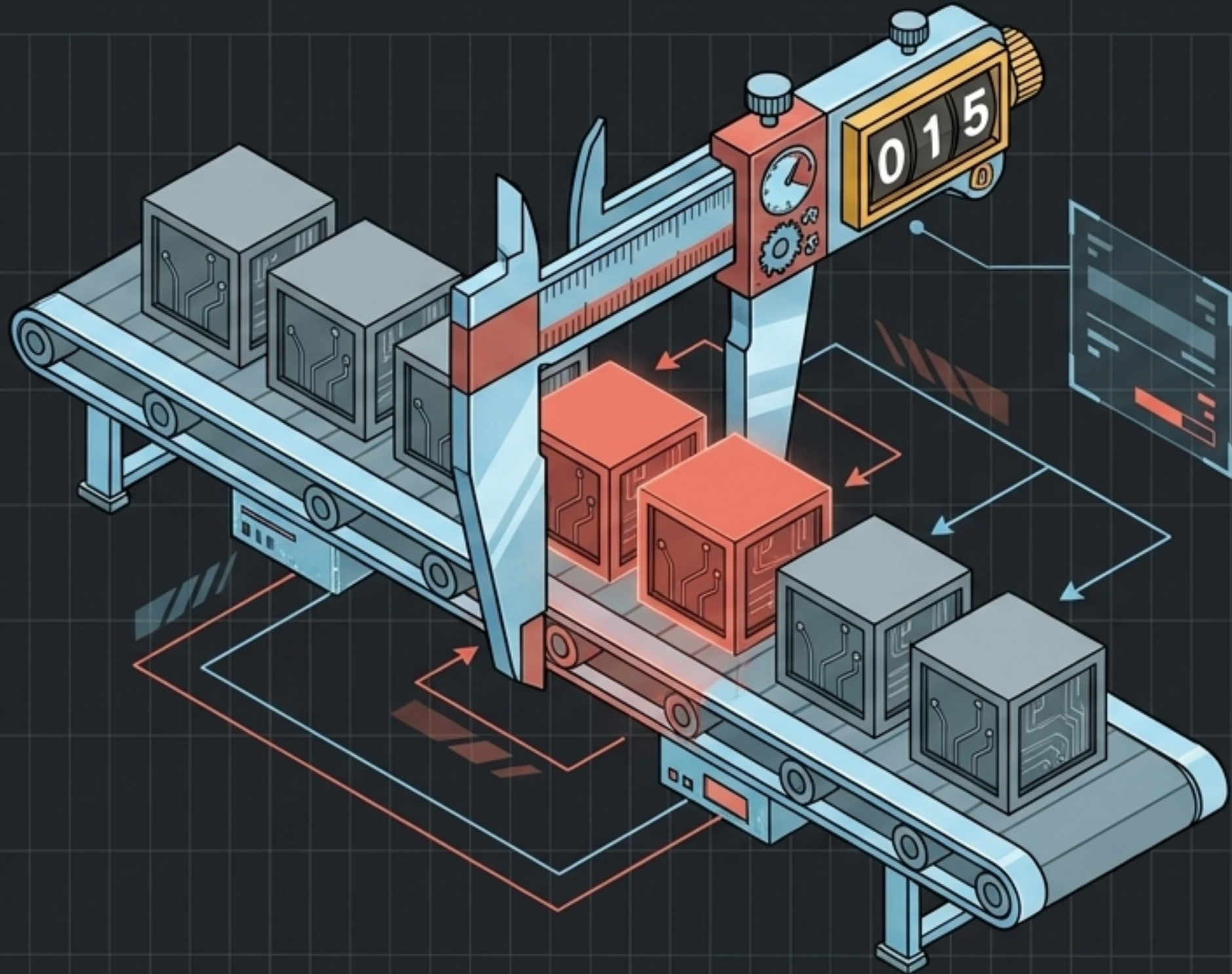
Peringatan Keras: Dilarang mutlak menggunakan fungsi bawaan Python seperti `.sort()`. Logika harus dibangun manual.

Mekanika Geser & Tukar (Odometer Perbandingan)

```
if arr[j] > arr[j+1]:  
    # TUKAR POSISI
```

Titik Hitung (Counter):

Setiap kali eksekusi program menyentuh baris evaluasi 'if arr[j] > arr[j+1]', variabel counter perbandingan harus ditambah 1, terlepas dari apakah data akhirnya ditukar atau tidak.



Mesin Pencari: Spesifikasi Linear Search



```
def linear_search(arr, target):
```

Tujuan: Mencari keberadaan nominal gaji tertentu (target) di dalam sistem.

Output Ganda: Wajib mengembalikan posisi indeks (jika ditemukan) dan total jumlah_langkah_pencarian. Jika tidak ditemukan, kembalikan -1.

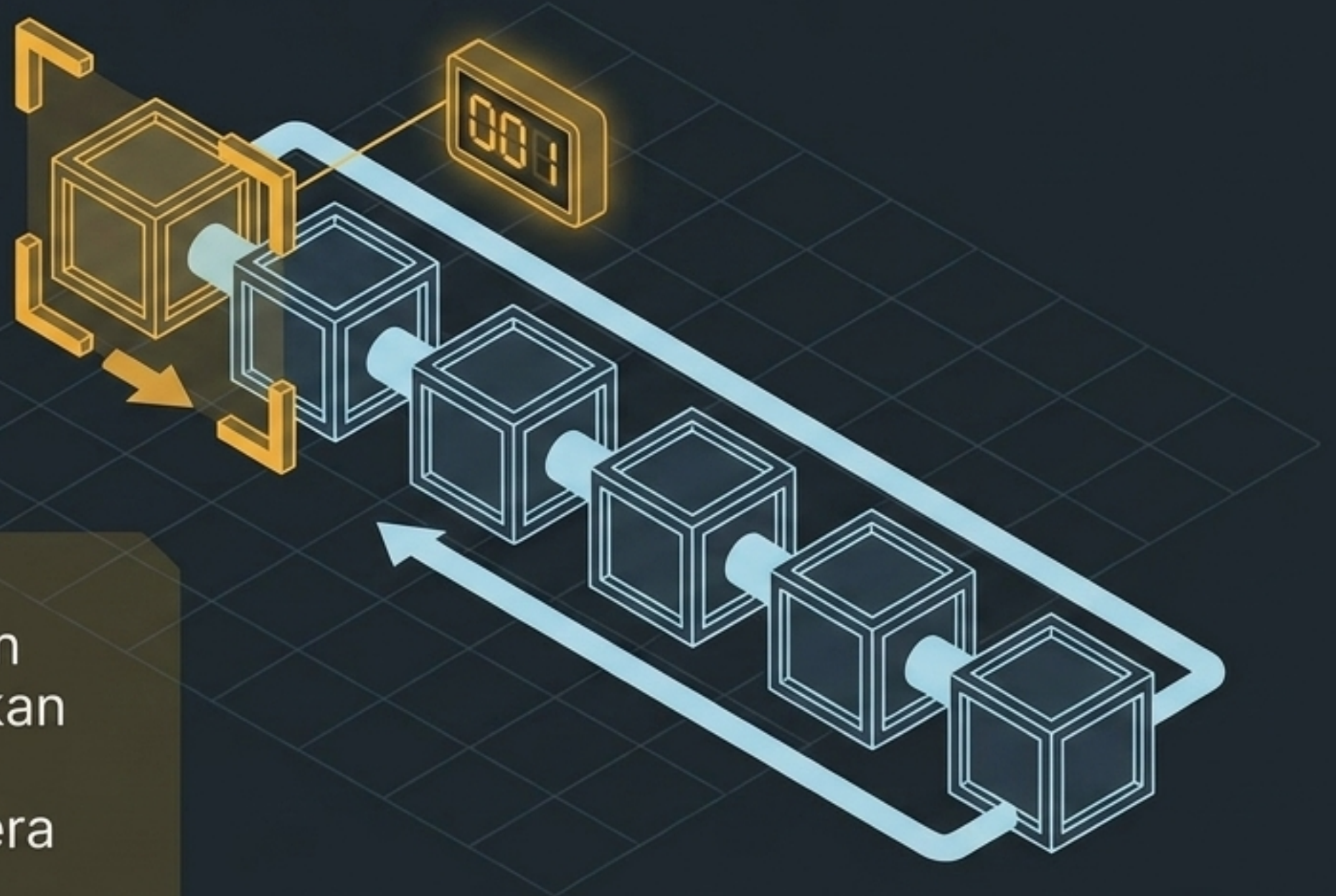
Peringatan Keras: Dilarang menggunakan operator *in-built* seperti 'in'. Pencarian harus melalui iterasi manual.

Mekanika Pemindaian (Target Akuisisi)

```
for i in range(len(arr)):  
    if arr[i] == target:  
        # TARGET DITEMUKAN
```

Titik Hitung (Counter):

Penambahan jumlah langkah dilakukan dilakukan setiap kali program melakukan validasi **'if arr[i] == target'**. Jika target ditemukan lebih awal, loop harus segera dihentikan (**break/return**).



Matriks Komparasi Algoritmik

	Bubble Sort	Linear Search
Karakteristik Loop	Bersarang (Nested Loop)	Tunggal (Single Loop)
Mekanisme Inti	Membandingkan dan menukar elemen bersebelahan	Mencocokkan setiap elemen dengan target dari kiri ke kanan
Kondisi Odometer	<code>if arr[j] > arr[j+1]</code>	<code>if arr[i] == target</code>
Kompleksitas Waktu	$O(n^2)$	$O(n)$

Orkestrasi Sistem: Fungsi main()

Tahap 1: Input Data

Eksekusi `input_gaji()` -> simpan dalam variabel `data_karyawan`.

1

Tahap 2: Pengurutan (Bubble Sort)

Oper `data_karyawan` ke `bubble_sort()` -> tangkap `data_urut` dan `total_banding`.

2

Tahap 3: Tampilkan Data Urut

Eksekusi `tampilkan_data()` untuk mencetak `data_urut`.

3

Tahap 4: Pencarian (Linear Search)

Minta input `target_gaji`, oper ke `linear_search()` -> tangkap `indeks` dan `total_langkah`.

4

Tahap 5: Analisis Akhir

Cetak Analisis Singkat ke terminal.

5

Format Audit Performa (Output Terminal)

=== ANALISIS SINGKAT ===

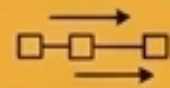
Algoritma sorting	: Bubble Sort
Kompleksitas sorting	: $O(n^2)$
Jumlah perbandingan	: [Hasil Counter]
Algoritma searching	: Linear Search
Kompleksitas searching	: $O(n)$
Jumlah langkah pencarian:	[Hasil Counter]
Algoritma dominan	: Sorting ($O(n^2)$)

Prinsip Dominasi Kompleksitas

Bubble Sort: $O(n^2)$

Linear Search: $O(n)$

Pencarian Linear (Stable $O(n)$)



Mekanisme: Loop tunggal.

Skala: Pertumbuhan konstan, aman untuk data besar.

Kompleksitas: $O(n)$

Pencarian Bersarang (Explosive $O(n^2)$)



Mekanisme: Loop bersarang.

Skala: Ledakan iterasi, runtuh pada data besar.

Kompleksitas: $O(n^2)$

Insight

Saat jumlah karyawan (n) bertambah, waktu eksekusi Bubble Sort akan meledak secara eksponensial karena struktur "nested loop". Sebaliknya, Linear Search hanya tumbuh secara konstan.

Waktu pemrosesan keseluruhan sistem akan selalu didikte oleh "mesin yang paling lambat", yaitu fungsi pengurutan (Sorting).

Aturan Emas Kelulusan Praktikum



Manual Engine Only: Penggunaan `.sort()` atau fungsi pencarian bawaan 'in' akan menganulir evaluasi logika Anda.



Isolasi Modul: Jika logika sorting dan searching tergabung dalam satu fungsi monolitik, nilai arsitektur modular akan hangus.



Akurasi Odometer: Pastikan indentasi penambahan counter (`count += 1`) berada tepat berdampingan dengan evaluasi 'if', bukan di luar loop.

Protokol Pengujian Terakhir (Time Attack)

